UNIVERSIDAD PRIVADA DEL VALLE

FACULTAD DE INFORMATICA Y ELECTRONICA

CARRERA DE INGENIERIA EN SISTEMAS INFORMÁTICOS



Actualización del Test Plan en un entorno ágil – PedidoHub

Estudiante:

- Luis Andrés Suárez Andrade

Asignatura: Aseguramiento de la Calidad del Software (SQA)

Semestre: Sexto

Grupo: A

Santa Cruz – Bolivia

Sistema	PedidoHub: gestión de restaurante (mesas, reservas, pedidos, facturación, reportes, roles y bitácora)
Autor	Luis Andrés Suárez Andrade (trabajo individual)
Repositorio	https://github.com/LuisSuarez11/PedidoHub_SQA
Qué pide la defensa	Mostrar el sistema, mostrar las pruebas realizadas con su evidencia y responder preguntas sobre calidad en el proceso de software, herramientas SQA, integración de la calidad en el proceso ágil y automatización

1. Presentación en 30 segundos

"PedidoHub es un sistema web para gestionar un restaurante. Mi trabajo fue asegurar su calidad dentro de un proceso ágil: definí un Test Plan con historias de usuario y criterios de aceptación, escribí las pruebas antes del código en las reglas críticas, automaticé 216 pruebas en tres niveles con xUnit, Selenium y Cypress, las ejecuto en integración continua con GitHub Actions y publico las métricas en un panel dentro del propio sistema. Encontré y corregí 6 defectos, cada uno con su prueba de regresión, y la cobertura de líneas de la lógica es de 81 %."

2. Qué hicimos y con qué resultado

Nivel	Qué verifica	Herramienta	Pruebas	Resultado
Unitarias	Validaciones, cálculos de total y de reportes	xUnit	46	46 pasan
Integración y aceptación (BDD)	Reglas de negocio, seguridad y datos a través de HTTP	xUnit + WebApplicationFactory + SQLite	134	134 pasan
E2E en navegador	Flujos de usuario reales	Selenium WebDriver	17	17 pasan
E2E en navegador	Flujos de usuario reales	Cypress	19	19 pasan
Total			**216**	**216 pasan (100 %)**

- **Cobertura de líneas** de la lógica de la aplicación (controladores, servicios, modelos): 81.3 % (1328 de 1633).
- **Defectos:** 6 encontrados y 6 corregidos (1 crítico, 3 mayores, 2 menores), con densidad de 3.7 por cada 1000 líneas ejecutables.
- **Tiempo de retroalimentación:** 13 segundos para las 180 pruebas unitarias y de integración.

3. Cómo hicimos las pruebas (respuesta a "digan cómo las hicieron")

- **Requisitos y criterios de aceptación.** Se definieron 17 historias de usuario (HU-01 a HU-17) con criterios en formato Dado / Cuando / Entonces. Es el punto de partida de cada prueba.
- **Diseño de casos con técnicas de caja negra.** Partición de equivalencia y análisis de valores límite. Ejemplos reales: NIT con 4, 5, 20 y 21 dígitos; capacidad de mesa 1, 20 y 21; precio de plato 0, 0.01, 10000 y 10000.01; cantidad de platos 1, 100 y 101; reserva dentro y fuera de la ventana de 2 horas.
- **Pruebas primero (TDD) en las reglas críticas.** Se escribió la prueba, se ejecutó en rojo (5 fallos esperados) y luego se implementó la regla hasta ponerla en verde. Evidencia: `Docs/evidencias/tdd-fase-roja.txt`.
- **Automatización por niveles.** Las reglas se prueban en el nivel más bajo posible (pirámide de pruebas): 46 unitarias, 134 de integración y 36 en navegador.
- **Datos de prueba controlados.** Cada prueba de integración crea una base de datos SQLite nueva con los datos iniciales; las E2E usan una base SQL Server dedicada (`PedidoHubDB_QA`) que se recrea limpia en cada ejecución.
- **Pruebas manuales documentadas.** Casos de prueba manuales con pasos, resultado esperado, resultado obtenido y captura (`Docs/CasosDePruebaManuales.md`), más sesiones exploratorias con objetivo definido.
- **Registro de defectos.** Cada hallazgo se registró con identificador, severidad, causa y acción correctiva, y se cerró con una prueba de regresión (`Docs/InformeAuditoria.md`).
- **Integración continua.** Cada *push* ejecuta compilación, pruebas y cobertura; luego las pruebas E2E contra SQL Server (`.github/workflows/ci.yml`).
- **Métricas y mejora continua.** Se actualizan con `scripts/cobertura.ps1` y se muestran en el Panel de calidad (`/Calidad`).

4. Evidencia: qué mostrar y dónde está

Si preguntan por...	Muestro	Ubicación
"¿Cómo saben que pasan las pruebas?"	Ejecución en vivo de `dotnet test`	Terminal, 13 s
"Quiero ver una prueba E2E"	Video de Cypress y capturas de Selenium	`Docs/evidencias/defensa/video_cypress/` y `capturas_selenium/`
"Reporte de resultados"	Archivos TRX y JUnit	`Docs/evidencias/defensa/`
"Cobertura"	Informe por archivo y panel	`cobertura.md` y `/Calidad`
"Integración continua"	Pestaña Actions del repositorio	https://github.com/LuisSuarez11/PedidoHub_SQA/actions
"Defectos y acciones correctivas"	Informe de auditoría y tabla de defectos	`Docs/InformeAuditoria.md` y `/Calidad`
"Pruebas manuales"	Casos manuales con capturas	`Docs/CasosDePruebaManuales.md` y `Docs/evidencias/manuales/`

Si preguntan por...	Muestro	Ubicación
"Test Plan y tablero"	Test Plan y backlog	`Docs/TestPlan_Agil.pdf`, `Docs/backlog-trello.csv`

La evidencia se regenera con un solo comando (sección 5), así que se puede mostrar creada en el momento.

5. Demostración en vivo (10 minutos)

Preparación (antes de empezar): SQL Server LocalDB activo, terminal abierta en la carpeta del proyecto, navegador con el repositorio y la pestaña Actions abiertos.

Minuto	Acción	Qué decir
0–2	`cd PedidoHub` y `dotnet run`; abrir `http://localhost:5158`	"Esta es la página pública; ya aquí se ven los indicadores de calidad"
2–4	Iniciar sesión como `admin` y abrir **Panel de calidad**	"Aquí están las métricas del Test Plan: pruebas, cobertura, defectos y trazabilidad de cada historia"
4–6	En otra terminal: `dotnet test PedidoHub.Tests`	"180 pruebas en unos 13 segundos: esto es la retroalimentación rápida"
6–8	Mostrar el video de Cypress y las capturas de Selenium	"Esto es una prueba de extremo a extremo con navegador real"
8–9	Mostrar GitHub Actions	"Cada cambio dispara el mismo conjunto de pruebas"
9–10	Mostrar `/Bitacora` y probar un acceso denegado con `mesero`	"Trazabilidad y control de acceso por rol"

Si algo falla en vivo: no improvisar. Decir "voy a mostrar la ejecución guardada" y abrir `Docs/evidencias/defensa/RESUMEN.md`, el video y la pestaña Actions.

Para regenerar toda la evidencia (unos 5 minutos):

```
powershell -ExecutionPolicy Bypass -File scripts/evidencia-defensa.ps1
```

6. Preguntas esperadas y respuestas

A. Calidad en el proceso de software

¿Qué es la calidad de software? Es el grado en que un producto satisface las necesidades de sus usuarios y cumple sus requisitos funcionales y no funcionales. La norma ISO/IEC 25010 la descompone en características: adecuación funcional, eficiencia, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad.

¿Cuál es la diferencia entre QA, QC y testing?

- **Aseguramiento de la calidad (QA/SQA):** actividades del *proceso* para prevenir defectos (estándares, revisiones, auditorías, Test Plan, métricas).
- **Control de calidad (QC):** actividades sobre el *producto* para detectar defectos (inspecciones y pruebas).

- **Testing:** parte del control de calidad; ejecutar el software para encontrar fallas. En el proyecto: el Test Plan, el DoR/DoD y la auditoría son QA; la ejecución de las pruebas es QC.

¿Verificación o validación?

- **Verificación:** ¿construimos el producto correctamente? (cumple la especificación). Ejemplo: la prueba que comprueba que el NIT tenga 5 a 20 dígitos.
- **Validación:** ¿construimos el producto correcto? (sirve al usuario). Ejemplo: la sesión exploratoria con el flujo de un mesero.

¿Error, defecto y falla? Un *error* es la equivocación humana; el *defecto* es lo que queda en el código o el requisito; la *falla* es el comportamiento incorrecto que se observa al ejecutar. Ejemplo: olvidar validar la fecha (error) → falta la validación (defecto) → se acepta una reserva en el pasado (falla).

¿Por qué la calidad se integra en todo el proceso y no solo al final? Porque el costo de corregir un defecto crece cuanto más tarde se detecta. Por eso se aplica *shift-left*: criterios de aceptación desde la historia, pruebas antes del código y ejecución continua.

¿Qué normas o modelos conoce? ISO/IEC 25010 (modelo de calidad del producto), ISO/IEC/IEEE 29119 (pruebas de software), IEEE 829 (documentación de pruebas, base del Test Plan tradicional) y CMMI e ISO 9001 como modelos de madurez del proceso. En este proyecto se aplicó ISTQB v4.0 y el modelo de pruebas ágiles de Crispin y Gregory.

¿Cuáles son los principios del testing? Las pruebas muestran la presencia de defectos, no su ausencia; las pruebas exhaustivas son imposibles; probar temprano; los defectos se agrupan; la paradoja del pesticida; las pruebas dependen del contexto; y la falacia de la ausencia de defectos. Ejemplo propio: cinco de los seis defectos aparecieron en las reglas de negocio y la seguridad (agrupamiento).

B. Herramientas SQA

¿Qué herramientas usaron y por qué?

Categoría	Herramienta	Uso en el proyecto
Pruebas unitarias e integración	xUnit y `WebApplicationFactory`	Pruebas rápidas de reglas, controladores y datos
Pruebas E2E	Selenium WebDriver y Cypress	Flujos de usuario en navegador real (se usan dos herramientas, como pidió la materia)
Cobertura	coverlet	Mide qué líneas ejecutan las pruebas
Integración continua	GitHub Actions	Ejecuta todo en cada cambio
Gestión del trabajo y defectos	Tablero Kanban (Trello/Jira)	Historias, defectos y deuda técnica
Monitoreo	Endpoint `/health`	Verifica aplicación y base de datos
Métricas	Panel de calidad propio	Muestra el estado de la calidad dentro del sistema

¿Qué otras herramientas existen y por qué no las usaron? TestLink y Squash TM (gestión de casos de prueba: se resolvió con el tablero y el Test Plan por ser un equipo de

una persona), SonarQube (análisis estático: queda como mejora), JMeter y k6 (carga: fuera de alcance), Postman (API: el sistema no tiene API JSON), OWASP ZAP (seguridad dinámica: mejora futura).

¿Por qué dos herramientas E2E? La materia pide al menos dos. Selenium comparte lenguaje y datos con el resto de la solución .NET; Cypress da un ciclo de depuración rápido, capturas y video. Se complementan, aunque cubren casos parecidos a propósito para compararlas.

¿Cómo eligieron las herramientas? Por compatibilidad con la tecnología (.NET), costo (todas gratuitas), integración con CI y curva de aprendizaje.

C. Integración de la calidad en el proceso ágil

¿Qué marco ágil usaron y cómo entra la calidad? Kanban con cadencia semanal y prácticas de XP. La calidad entra con: criterios de aceptación en cada historia, *Definition of Ready* para empezar, *Definition of Done* para cerrar (pruebas en verde, cobertura, cero defectos críticos), TDD, integración continua y retrospectiva con métricas.

¿Qué es el DoR y el DoD? El *Definition of Ready* indica cuándo una historia está lista para empezar (redactada, con criterios de aceptación, prioridad y datos de prueba). El *Definition of Done* indica cuándo está terminada (pruebas escritas y en verde, cobertura mínima, sin defectos graves, CI en verde y documentación actualizada).

¿Quién es responsable de la calidad en un equipo ágil? Todo el equipo. En este proyecto de una persona, los roles (PO, desarrollador, QA, facilitador) se ejercieron por separado y se compensó la falta de revisión independiente con TDD y con el CI como verificador objetivo.

¿Qué es TDD, BDD y ATDD? TDD: se escribe la prueba antes del código (rojo, verde, refactor). BDD: las pruebas describen comportamiento con Dado/Cuando/Entonces. ATDD: el criterio de aceptación definido con el negocio se convierte en prueba automática antes de implementar. En el proyecto: TDD en los defectos DEF-01 a DEF-05; BDD en la nomenclatura de las pruebas de aceptación.

¿Qué son los cuadrantes de pruebas ágiles?

Cuadrante	Enfoque	En PedidoHub
Q1: técnicas, apoyan al equipo	Pruebas unitarias y de componentes	46 unitarias de xUnit
Q2: de negocio, apoyan al equipo	Pruebas funcionales y de aceptación	134 de integración y los E2E
Q3: de negocio, critican el producto	Exploratorias, usabilidad, aceptación del usuario	Casos manuales y sesiones exploratorias
Q4: técnicas, critican el producto	Rendimiento, carga, seguridad	Parcial: seguridad básica (antiforgery, roles, cabeceras); carga pendiente

¿Cómo se integra la calidad con CI/CD? Con un pipeline de dos etapas: compilación, pruebas y cobertura en cada *push*; luego E2E contra SQL Server. Un pipeline en rojo significa que no se hace *merge*.

¿Qué métricas usan? Tasa de éxito (100 %), cobertura de líneas (81.3 %), densidad de defectos (3.7 por KLOC), defectos corregidos (6 de 6), tiempo de la suite (13 s) y *throughput* del sprint (23 tarjetas). Se muestran en `/Calidad`.

D. Automatización

¿Qué automatizaron primero y por qué? El cálculo de totales y las reglas de facturación, porque involucran dinero y cambian con frecuencia, y el inicio de sesión y los roles, porque protegen todo el sistema.

¿Qué no automatizaron y por qué? Usabilidad, diseño visual, redacción de mensajes, exploración del plano de mesas, pruebas de carga y aceptación con usuario real: necesitan criterio humano o no son rentables aún.

¿Qué desventaja tiene automatizar todo por la interfaz? Son lentas, frágiles ante cambios visuales, difíciles de diagnosticar y caras de mantener. Por eso solo 36 de 216 pruebas usan el navegador.

¿Cómo evitan que las pruebas se vuelvan difíciles de mantener? Una prueba por regla en el nivel más bajo, código de apoyo compartido, pruebas independientes, selectores estables, nombres descriptivos y corrección inmediata de las pruebas rojas.

¿Cómo saben que automatizar valió la pena? Por los defectos detectados antes de liberar (seis, todos con regresión), el tiempo de retroalimentación (13 s frente a una regresión manual de aproximadamente una hora, estimada) y el bajo costo de mantenimiento.

¿Qué harían si la interfaz cambia mucho? Mover la verificación a niveles inferiores, dejar pocas pruebas E2E de flujos críticos y centralizar selectores y pasos comunes.

E. Preguntas sobre las pruebas del sistema

¿Cuántas pruebas hicieron y qué encontraron? 216 automatizadas más casos manuales. Encontraron 6 defectos: reserva en fecha pasada, dos pedidos abiertos en la misma mesa, factura con fecha anterior al pedido, falta de endpoint de salud, falta de cabeceras de seguridad y ausencia de autenticación.

¿Cómo se ejecutaron las pruebas? Las unitarias y de integración con `dotnet test`; las E2E con `scripts/run-e2e.ps1` (levanta la aplicación con una base limpia, ejecuta Selenium y Cypress y apaga todo); la evidencia completa con `scripts/evidencia-defensa.ps1`.

¿Cómo se comprobó que una prueba es buena? Cada regla se verificó viendo la prueba **fallar primero** y luego pasar (TDD). Una prueba que nunca se vio en rojo puede no estar comprobando nada.

¿Por qué las pruebas de integración usan SQLite y no SQL Server? Por velocidad y aislamiento: una base nueva por prueba en memoria. El riesgo es que el comportamiento difiera (por ejemplo, SQLite no suma decimales), así que se adaptó el modelo de pruebas y las E2E y el CI corren contra SQL Server real.

¿Qué no probaron? Rendimiento y carga, pruebas de penetración, accesibilidad formal, compatibilidad con varios navegadores (solo Chrome y el motor de Cypress) y pruebas en dispositivos móviles reales. Se registran como limitaciones y mejoras.

¿El 81 % de cobertura significa que el 81 % del sistema es correcto? No. La cobertura mide qué líneas se ejecutaron, no si se verificó lo correcto. Sirve para detectar zonas sin pruebas (por ejemplo `TelefonosController`, con 51 %), no para garantizar calidad.

¿Por qué no 100 % de cobertura? Porque el costo de cubrir el último tramo supera su valor; se prioriza por riesgo. La meta fue 80 % en la lógica.

¿Las pruebas encontraron todos los errores? No: las pruebas muestran la presencia de defectos, no su ausencia. Quedan riesgos conocidos, registrados como deuda técnica.

¿Cómo trabajaron si el equipo era de una persona? Se ejercieron los roles por separado y se compensó la falta de revisión independiente con TDD, integración continua y auditoría con lista de verificación. Es una limitación declarada en el Test Plan.

¿Qué mejorarían con más tiempo? Análisis estático (SonarQube), pruebas de carga con k6, protección de la rama principal en GitHub, bloqueo por intentos fallidos de inicio de sesión y pruebas en más navegadores.

7. Calidad del producto según ISO/IEC 25010

Característica	Cómo se cubre en PedidoHub	Prueba o evidencia
Adecuación funcional	Reglas de reservas, pedidos y facturación	`ReglasDeNegocioTests`
Seguridad	Login, roles, claves con hash, antiforgery, cabeceras, cookie HttpOnly	`SeguridadTests`, `NavegacionTests`
Fiabilidad	Endpoint `/health`, validaciones, bitácora	`Health_*`, `BitacoraTests`
Usabilidad	Plano de mesas, búsqueda `Ctrl + K`, vista móvil	Casos manuales y E2E-11
Eficiencia	Suite de 13 s; sin pruebas de carga todavía	Limitación declarada
Mantenibilidad	Estructura por capas, pruebas, cobertura 81.3 %	`cobertura.md`
Portabilidad	Corre en Windows y en el CI (Linux)	GitHub Actions
Compatibilidad	Solo probado en Chrome	Limitación declarada

8. Glosario rápido

- **Regresión:** volver a probar lo que ya funcionaba tras un cambio.
- **Prueba de humo:** verificación rápida de que lo esencial responde (todas las pantallas cargan).
- **Caja negra / caja blanca:** probar sin mirar el código / mirando su estructura.
- **Valor límite:** probar en los bordes de un rango (4, 5, 20, 21 dígitos).
- **Prueba inestable (*flaky*):** a veces pasa y a veces falla sin cambios en el código.
- **Prueba de aceptación:** comprueba que se cumple el criterio acordado con el negocio.

- **Shift-left:** adelantar las actividades de calidad al inicio del proceso.
- **Deuda técnica:** trabajo pendiente que se acepta por ahora y se registra para después.

9. Lista de verificación para el día anterior

- ☐ Ejecutar `scripts/evidencia-defensa.ps1` y comprobar que `Docs/evidencias/defensa/RESUMEN.md` muestra todas las pruebas en verde.
- ☐ Iniciar la aplicación con `dotnet run` y entrar con `admin`, `cajero` y `mesero`.
- ☐ Abrir la pestaña Actions del repositorio y comprobar que el último CI esté en verde.
- ☐ Leer el Test Plan, el informe de auditoría y esta guía.
- ☐ Ensayar la demostración de 10 minutos con cronómetro.
- ☐ Tener a mano el PDF del Test Plan y los casos manuales con sus capturas.
- ☐ Cargar el portátil y tener un plan B (video y capturas guardados).